Enemy wave memory console

Chapter 2 · Practical Lab 3 · Dynamic memory and object lifetime

In this lab, you will build a small enemy-wave model that owns a dynamically allocated array. You will first practise one raw dynamic object and one raw object array, then move the array into an owning class whose destructor performs the cleanup.

This is not a copy-the-code exercise. The class interfaces, behaviour rules and expected console output are supplied. You will write most member-function definitions, loops and validation logic yourself, one small stage at a time.

Duration: 120 minutes. Start from an empty Visual Studio project. Build after every small milestone rather than writing the whole program before the first run.

> **How to use this sheet.** Read the behaviour first, write the definition yourself, then build and compare with the published output. Hints are provided for new syntax, but complete function bodies are usually not supplied.

Before you begin

Use a Windows PC with **Visual Studio 2026**, the **Desktop development with C++** workload, Platform Toolset **v145**, C++17, **/W4**, **Debug** and **x64**.

Prerequisites: the class/object, constructor, const-access, fixed-array, forward-declaration and basic pointer skills from the previous Chapter 2 work.

What you will practise

- creating and releasing one dynamically allocated object with new and delete;
- creating and releasing an array of class objects with new[] and delete[];
- distinguishing a pointer variable from the object or array it refers to;
- writing a class that owns one dynamic array and releases it in a destructor;
- using a forward declaration for a pointer member and a complete type in the implementation;
- validating indices before accessing a dynamic array;
- tracing how normal scope exit triggers an owning class destructor; and
- explaining why accidental copying is disabled for a raw-pointer owner.

Not required in this lab

You are not required to implement smart pointers, exception handling, custom allocators, inheritance, templates, STL containers, pointer arithmetic, a production memory manager or deliberate unsafe operations. Do not run double-delete, use-after-delete, out-of-bounds or mismatched-delete experiments.

The problem

A game spawns a small wave of enemies. Each enemy starts with 100 health. The size of the wave is chosen while the program is running, so the wave stores its enemies in a dynamic array. The EnemyWave object owns that array: it creates the enemies, provides safe operations for them, and releases the array when the wave is destroyed.

The central question is not simply “where is the pointer?” Ask instead: what was allocated, who owns it, and which operation ends its lifetime.

Files you will create

File	Responsibility
Main.cpp	Creates the demonstration objects, calls public interfaces and prints the published checks.
Enemy.h / Enemy.cpp	Defines one enemy and its bounded health behaviour.
EnemyWave.h / EnemyWave.cpp	Defines the owner of one dynamic Enemy array and releases it in its destructor.

What the finished program will show

```
Enemy wave memory lab

Single dynamic enemy
Starting health: 100
After 30 damage: 70

Raw dynamic array
Enemy 0 health: 75
Enemy 1 health: 0
Enemy 2 health: 40
Alive enemies: 2
Total health: 115

Owned wave
Enemies: 3
Enemy 0 health: 75
Enemy 1 health: 0
Enemy 2 health: 40
Alive enemies: 2
Total health: 115

Invalid wave
Enemies: 0
Total health: 0
```

Your two-hour route

Stage	What you will do	Time
01	Create the project and files from scratch	15 min
02	Define Enemy and manage one dynamic object	20 min
03	Allocate, use and release a dynamic Enemy array	20 min
04	Create EnemyWave as the owner of a dynamic array	25 min
05	Implement wave operations and boundary checks	25 min
06	Final run, explanation and cleanup review	15 min

01 · Create the project from scratch

Create a new C++ Empty Project named EnemyWaveMemoryLab. Use the same solution name unless your lecturer tells you otherwise.

Step 1 - Choose Debug and x64 in the Visual Studio toolbar.

Step 2 - For All Configurations, set Platform Toolset to v145, C++ Language Standard to ISO C++17, and Warning Level to Level 4 (/W4).

Step 3 - Add Main.cpp under Source Files.

Start with one working line

```
#include <iostream>

int main()
{
    std::cout << "Enemy wave memory lab\n";
    return 0;
}
```

Build the solution and run it before creating the classes.

Expected output

```
Enemy wave memory lab
```

Create the remaining files now

Header Files	Source Files
Enemy.h	Enemy.cpp
EnemyWave.h	EnemyWave.cpp

Checkpoint 1. Do not continue until the empty project builds and runs and all five files appear in the project. This gives you a known working starting point.

02 · Define one Enemy

The Enemy class is deliberately small. Its job is to store one health value and enforce the same bounded-damage rule you have already seen. This keeps the new learning focused on lifetime and ownership.

Create Enemy.h

```
#ifndef ENEMY_H
#define ENEMY_H

class Enemy
{
public:
    static constexpr int MAX_HEALTH{100};

    Enemy() = default;
    void takeDamage(int t_amount);
    int health() const;
    bool isAlive() const;

private:
    int m_health{MAX_HEALTH};
};

#endif
```

Write the definitions yourself

Open Enemy.cpp, include Enemy.h, and implement these definitions. Build after each one.

Definition	Required behaviour
takeDamage(int t_amount)	Zero or negative damage changes nothing. Positive damage reduces health, but health must never go below 0.
health() const	Return the current health value.
isAlive() const	Return true exactly when health is greater than 0.

Hint. For takeDamage, decide whether the requested amount reaches or exceeds the remaining health before subtracting. You should not need a loop.

Manage one dynamic Enemy

In Main.cpp, include Enemy.h. Under a heading called Single dynamic enemy, write the following ownership sequence yourself:

- declare an Enemy* named pEnemy and use new to create one default Enemy;
- print its starting health using the pointer access operator ->;
- apply 30 damage and print the new health;
- release that one object with the matching single-object delete; and
- set pEnemy to nullptr after the release so that this local variable no longer stores the old address.

Expected output for this stage

```
Single dynamic enemy
Starting health: 100
After 30 damage: 70
```

Ownership check. There is one pointer variable and one separately allocated Enemy. The pointer leaving scope would not release that Enemy by itself. The delete is required because this code owns the allocation.

03 · Allocate a dynamic Enemy array

Now repeat the lifetime reasoning for an array. This time the number of enemies is stored in a variable rather than written directly into the type.

Build the array step by step

Add a temporary Raw dynamic array experiment to Main.cpp. You will later keep the same behaviour inside EnemyWave.

1. Declare int enemyCount{3}.
2. Declare an Enemy* named pEnemies and allocate enemyCount enemies with array new[]. Each element should use the default Enemy constructor.
3. Apply 25 damage to element 0, 100 damage to element 1, and 60 damage to element 2.
4. Use a loop to calculate the number of living enemies.
5. Use a loop to calculate the total remaining health.
6. Print each enemy health, then the alive count and total health.
7. Release the whole object array with the matching array delete[] and set pEnemies to nullptr.

> **Important.** Do not write delete pEnemies[i] for these elements. They are elements of one array allocation, not three separate new-expressions.

Expected output

```
Raw dynamic array
Enemy 0 health: 75
Enemy 1 health: 0
Enemy 2 health: 40
Alive enemies: 2
Total health: 115
```

Objects and pointer slots are different

Expression	What it creates	Matching release
`new Enemy[3]`	Three Enemy objects in one array allocation	`delete[]` the base pointer
`new Enemy*[3]{}`	Three Enemy* pointer slots, initially nullptr; zero Enemy objects	`delete[]` the slot-array pointer

> **Recognition check.** You do not need to implement the pointer-slot version in this lab. Be able to explain why the star changes the array element type and therefore changes what was constructed.

04 · Make EnemyWave own the array

The raw array experiment worked, but Main.cpp had to remember the cleanup. We will now move that responsibility into a class. EnemyWave owns one dynamic Enemy array from construction until destruction.

Create EnemyWave.h

```
#ifndef ENEMY_WAVE_H
#define ENEMY_WAVE_H

class Enemy;

class EnemyWave
{
public:
    explicit EnemyWave(int t_count);
    ~EnemyWave();

    EnemyWave(EnemyWave const&) = delete;
    EnemyWave& operator=(EnemyWave const&) = delete;

    int count() const;
    bool damageEnemy(int t_index, int t_amount);
    int healthAt(int t_index) const;
    int aliveCount() const;
    int totalHealth() const;

private:
    static constexpr int MAX_ENEMIES{5};
    int m_count{0};
    Enemy* m_enemies{nullptr};
};

#endif
```

Why only a forward declaration here? EnemyWave.h stores an Enemy pointer, not an Enemy object by value. The header only needs the class name. EnemyWave.cpp will include Enemy.h because allocation and member calls require the complete Enemy definition.

Start EnemyWave.cpp

```
#include "EnemyWave.h"
#include "Enemy.h"

EnemyWave::EnemyWave(int t_count)
{
    // Validate t_count, store the accepted count,
    // and allocate an Enemy array only when count > 0.
}

EnemyWave::~EnemyWave()
{
    // Release the array owned by this EnemyWave.
}
```

Write the constructor logic yourself

Rule	Required definition
Accepted count	0 to 5 inclusive.
Invalid count	Leave m_count as 0 and m_enemies as nullptr.
Positive accepted count	Store the count and allocate exactly that many Enemy objects with new[].
Count 0	No array allocation is needed.

Hint. The in-class member initialisers already give m_count and m_enemies safe starting values. Validate first. Only perform new Enemy[m_count] after you know the count is valid and positive.

Write the destructor

The destructor owns exactly one cleanup responsibility: release the dynamic array created by this EnemyWave. Match the array allocation form. You do not need a loop and you do not delete m_count or the pointer variable itself.

> **Copy protection.** The two = delete declarations deliberately prevent a second EnemyWave from receiving a shallow copy of the owning pointer. You are not being asked to implement deep copying or move operations in this practical.

05 · Implement the EnemyWave operations

Complete one member-function definition at a time in EnemyWave.cpp. Build after each definition. The public interface is already fixed; your job is to turn each behaviour statement into clear C++ logic.

Function	Required behaviour
count() const	Return m_count.
damageEnemy(index, amount)	If the index is invalid, return false. Otherwise call takeDamage on that array element and return true.
healthAt(index) const	Return -1 for an invalid index. Otherwise return that enemy's health.
aliveCount() const	Loop over the current array and count enemies whose isAlive() query returns true.
totalHealth() const	Loop over the current array and add each health value to a running total.

> **Index rule.** An index is valid only when it is at least 0 and strictly less than m_count. Validate before using m_enemies[t_index].

Check the owned wave

In Main.cpp, construct EnemyWave wave{3};. Apply the same three damage amounts used in the raw-array experiment: 25 to index 0, 100 to index 1 and 60 to index 2. Print the wave count, each enemy health, alive count and total health.

Expected output

```
Owned wave
Enemies: 3
Enemy 0 health: 75
Enemy 1 health: 0
Enemy 2 health: 40
Alive enemies: 2
Total health: 115
```

Check an invalid wave size

Construct EnemyWave invalid{8};. The constructor should reject this size without allocating an array. Print its count and total health.

```
Invalid wave
Enemies: 0
Total health: 0
```

One small boundary check

Temporarily call wave.damageEnemy(9, 10). Predict the result before running. It should return false, the three stored enemy health values should remain unchanged, and the program must not access outside the array. Remove the temporary check afterwards.

> **Destructor check.** You do not need to add printing inside the destructor. If you want to observe it, set a breakpoint on EnemyWave::~EnemyWave and run normally. Leaving main or leaving a nested scope destroys the local EnemyWave and runs its cleanup.

06 · Put Main.cpp into its final form

Main.cpp is only the driver. Keep the print statements you developed during the earlier stages. The skeleton below shows the required objects, variables, operations and cleanup; use the published final expected output to decide what still needs to be printed.

```
int main()
{
    // Print the lab heading.

    // 1. One dynamic Enemy: allocate, damage, print, delete, nullptr.
    Enemy* pEnemy{new Enemy{}};
    pEnemy->takeDamage(30);
    // Print starting and final health before cleanup.
    delete pEnemy;
    pEnemy = nullptr;

    // 2. Raw Enemy array: allocate 3, apply 25/100/60 damage,
    //    calculate alive count and total, print, then delete[].
    int enemyCount{3};
    Enemy* pEnemies{new Enemy[enemyCount]};
    // Your existing raw-array operations and output go here.
    delete[] pEnemies;
    pEnemies = nullptr;

    // 3. EnemyWave owner: repeat the same damage through its interface.
    EnemyWave wave{3};
    wave.damageEnemy(0, 25);
    wave.damageEnemy(1, 100);
    wave.damageEnemy(2, 60);
    // Print count, health values, alive count and total.

    // 4. Invalid wave: no allocation, count 0, total 0.
    EnemyWave invalid{8};
    // Print the published invalid-wave values.

    return 0;
}
```

Final expected output

```
Enemy wave memory lab

Single dynamic enemy
Starting health: 100
After 30 damage: 70

Raw dynamic array
Enemy 0 health: 75
Enemy 1 health: 0
Enemy 2 health: 40
Alive enemies: 2
Total health: 115

Owned wave
Enemies: 3
Enemy 0 health: 75
Enemy 1 health: 0
Enemy 2 health: 40
Alive enemies: 2
Total health: 115

Invalid wave
Enemies: 0
Total health: 0
```

If something is not working

Symptom	First thing to inspect
Unresolved external symbol	Check that the declaration and definition signatures match and that the correct .cpp file is included in the project.
Enemy is an incomplete type in EnemyWave.cpp	Include Enemy.h in EnemyWave.cpp before using new Enemy[...] or calling Enemy members.
Raw array works but EnemyWave crashes	Check count validation, allocation only for a positive valid count, and index validation before array access.
Health falls below zero	Repair Enemy::takeDamage; do not compensate by hard-coding output in Main.cpp.
Invalid wave reports count 8	The constructor accepted a value outside the 0-5 contract.
Cleanup uses delete instead of delete[]	Match the allocation history: new Enemy[...] requires delete[]. Do not run the mismatched form as an experiment.

Short understanding check

Question	What your explanation should distinguish
1. How many objects exist after Enemy* pEnemy = new Enemy{};?	One local pointer variable and one separately allocated Enemy object.
2. Why does new Enemy[3] require delete[]?	It created one array allocation containing three Enemy elements; array cleanup destroys the elements and releases that block.
3. Why can EnemyWave.h forward-declare Enemy?	The member stores only an Enemy pointer. The full Enemy definition is needed later where allocation and member calls occur.
4. What does the EnemyWave destructor own?	Only the dynamic Enemy array created for that wave; not borrowed objects or ordinary embedded scalar members.
5. Why are copy operations deleted?	A default raw-pointer copy would copy the address rather than duplicate the allocation, producing conflicting ownership.
6. How many Enemy objects are created by new Enemy*[3]{}?	Zero; it creates three pointer slots initialised to nullptr.

Final checklist

☐ The EnemyWaveMemoryLab solution and project were created from scratch.

☐ The project uses v145, C++17, /W4, Debug and x64.

☐ Enemy health never falls below zero.

☐ The single dynamic Enemy is released with delete and the local pointer is cleared.

☐ The raw Enemy array is released with delete[].

☐ EnemyWave.h forward-declares Enemy and stores an Enemy* pointer member.

☐ EnemyWave.cpp includes Enemy.h before allocating or using Enemy objects.

☐ EnemyWave accepts counts 0-5 and rejects other values without allocating.

☐ EnemyWave::~EnemyWave releases its owned array with delete[].

☐ All array indices are validated before use.

☐ The copy constructor and copy assignment remain deleted.

☐ The final console output matches the published output.

☐ Temporary boundary-check code has been removed and the final project builds cleanly.

Optional stretch · add empty()

Only attempt this after the core output matches. Add this public query to EnemyWave:

```
bool empty() const;
```

Write the definition yourself using only the stored count. It should return true for EnemyWave{0} and rejected sizes such as EnemyWave{8}, and false for EnemyWave{3}. The query must not allocate, delete or change any enemy.

Lecture and tutorial alignment

Lab stage	Primary source alignment
02 · One dynamic object	Lecture 3 slides 2-7: automatic/dynamic lifetime, new/delete, ownership and aliases; Tutorial 3 stages 01-02.
03 · Dynamic object array	Lecture 3 slides 10-11 and 16-20: new[], class arrays, object arrays versus pointer arrays and matching cleanup; Tutorial 3 stages 02-05.
04-05 · EnemyWave owner	Lecture 3 slides 14-15 and 21-24: embedded versus separate storage, destructors, RAII and copy protection; Tutorial 3 stages 06-07.
06 · Lifetime explanation	Lecture 3 slides 25-26: identify what exists, who owns it and when the lifetime ends; Tutorial 3 final evidence model.

Deliberate scope boundary. The tutorial also explores exception cleanup, borrowed pointer arrays, separately owned targets and a smart-pointer extension. Those are useful guided-practice topics, but this independent practical keeps the core implementation smaller: one dynamic object, one dynamic object array, and one class that owns and releases that array.